

THE MATHEMATICS BEHIND THE bCCNN

The ccODP model, the network and its chain-ladder start

Study notes on the bCCNN strand of the research assignment

Prepared with the generative artificial intelligence tool Claude (Anthropic)
as study material for the BComHons research assignment
Department of Statistics and Actuarial Science
Stellenbosch University

Not for submission for assessment

October 2026

Note on the preparation of these notes

These notes were prepared with the generative artificial intelligence (AI) tool Claude (Anthropic) at the request of the student, as study material on the blended cross-classified neural network (bCCNN) strand of the research assignment. They explain the mathematics of the model, the code in the student's repository (Masters-Research-R) and the literature that the code follows.

The notes are not the student's original work and are not intended for submission for assessment. Section 2.6.3 of the Research Assignment Guide (Department of Statistics and Actuarial Science, 2026: 7-8) prohibits the use of AI tools to generate content that is submitted for assessment and requires a declaration of any use of such tools. For this reason the plagiarism declaration and the declaration on the use of artificial intelligence tools of the Department template are not included. The layout follows the technical requirements in Chapter 5 of the Guide and the Department template.

Abstract

This report sets out, formula by formula, the mathematics behind the blended cross-classified neural network that the research code fits to a 20 by 20 annual claims triangle. It starts from the cross-classified over-dispersed Poisson model on which the chain ladder rests, derives its maximum likelihood equations and shows, through the marginal-total property, that the fitted model reproduces the chain-ladder reserve. It then describes the neural network that is wrapped around this model – the frozen embeddings, the skip connection, the three hidden layers with dropout and the exponential output neuron – together with the loss that Keras minimises and its relation to the Poisson deviance.

The gradients of the loss are written out in full. At the prescribed starting point the network reproduces the chain-ladder means exactly, only the output weights of the hidden features have a non-zero gradient, and the first optimiser step moves each of these weights by a fixed amount that does not depend on the size of the gradient. The report further treats the reserves, the dispersion of the network, the diagnostics computed from the fitted means and the handling of periods without payments, and it maps every formula to the place in the code where it is computed. A numerical check on a synthetic triangle, carried out independently of Keras, confirms the formulas to machine precision.

Key words: Claims reserving; Chain ladder; Over-dispersed Poisson model; Neural networks; Embedding layers; Poisson deviance; Gradient descent; RMSprop; Dispersion

Table of contents

Note on the preparation of these notes.....	ii
Abstract.....	iii
List of tables.....	vii
List of figures.....	viii
List of abbreviations and/or acronyms.....	ix
CHAPTER 1 INTRODUCTION.....	1
1.1 INTRODUCTION.....	1
1.2 THE CLAIMS TRIANGLE.....	2
1.3 SCOPE AND LIMITATIONS.....	2
1.4 VERIFICATION OF THE MATHEMATICS.....	3
1.5 OUTLINE OF THE DOCUMENT.....	3
CHAPTER 2 THE CROSS-CLASSIFIED OVER-DISPERSED POISSON MODEL.....	4
2.1 INTRODUCTION.....	4
2.2 ASSUMPTIONS.....	4
2.3 IDENTIFICATION.....	4
2.4 THE POISSON DEVIANCE.....	5
2.5 THE MAXIMUM LIKELIHOOD EQUATIONS.....	5
2.6 HOW GLM() SOLVES THEM: IWLS.....	6
2.7 THE CCODP IS THE CHAIN LADDER.....	7
2.8 DISPERSION.....	8
2.9 THE UNIT OF THE PAYMENTS.....	8
2.10 PERIODS WITHOUT PAYMENTS.....	9
2.10.1 What happens.....	9
2.10.2 Why it is harmless for the chain ladder.....	9
2.10.3 What Paper C and Härkönen do.....	9
2.10.4 What the code does.....	9
2.11 SUMMARY.....	10
CHAPTER 3 THE bCCNN NETWORK AND ITS LOSS.....	11

3.1	INTRODUCTION.....	11
3.2	EMBEDDING LAYERS.....	11
3.3	THE SKIP CONNECTION.....	12
3.4	THE FEED-FORWARD PART.....	12
3.5	THE OUTPUT NEURON.....	13
3.6	DROPOUT.....	14
3.7	THE INITIALISATION.....	14
3.8	THE LOSS AS KERAS SEES IT.....	15
3.8.1	Keras's Poisson loss and the deviance.....	15
3.8.2	What is recorded after every step.....	15
3.9	SUMMARY.....	16
CHAPTER 4 GRADIENTS AND THE START OF GRADIENT DESCENT.....		17
4.1	INTRODUCTION.....	17
4.2	DERIVATIVE IN THE OUTPUT.....	17
4.3	OUTPUT LAYER.....	17
4.4	HIDDEN LAYERS (BACK-PROPAGATION).....	17
4.5	PROPERTY 2: THE GRADIENT AT THE START.....	18
4.6	PROPERTY 3: THE SIZE OF THE FIRST STEPS.....	19
4.7	SUMMARY.....	20
CHAPTER 5 RESERVES, DISPERSION AND DIAGNOSTICS.....		21
5.1	INTRODUCTION.....	21
5.2	FROM THE FITTED MEANS TO THE RESERVES.....	21
5.3	DISPERSION OF THE BCCNN.....	21
5.4	DIAGNOSTICS COMPUTED FROM THE MEANS.....	22
5.4.1	Pearson residuals.....	22
5.4.2	Cumulative development factors.....	22
5.4.3	Relative difference.....	22
5.5	SUMMARY.....	23
CHAPTER 6 IMPLEMENTATION AND VERIFICATION.....		24
6.1	INTRODUCTION.....	24

6.2	WHERE EACH FORMULA IS IN THE CODE.....	24
6.3	NUMERICAL CHECK OF THE FORMULAS.....	25
6.4	SUMMARY AND CONCLUSIONS.....	26
References.....		28
APPENDIX A LIST OF SYMBOLS.....		29

List of tables

Table 3.1 Parameters of the network.....	13
Table 3.2 The history recorded by <code>fit_bcnn()</code>	16
Table 6.1 Location of the formulas in the code.....	24
Table 6.2 Results of the numerical check (amounts in millions of the synthetic data).....	26
Table A.1 Symbols used in this report.....	29

List of figures

Figure 3.1 The network of <code>bccnn_model()</code> with the Keras layer names.....	11
Figure 4.1 Norms of the gradient by parameter group over the first RMSprop steps in the numerical check	19

List of abbreviations and/or acronyms

bCCNN	Blended cross-classified neural network
CANN	Combined actuarial neural network
ccODP	Cross-classified over-dispersed Poisson
GLM	Generalised linear model
IWLS	Iteratively weighted least squares
LoB	Line of business
MLE	Maximum likelihood estimate
RMSprop	Root mean square propagation
SGD	Stochastic gradient descent
SSRN	Social Science Research Network
TW	Trainable input weights (label used by Härkönen)

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

This report writes out, formula by formula, what the code of the blended cross-classified neural network (bCCNN) computes on one 20×20 annual claims triangle. The bCCNN combines the cross-classified over-dispersed Poisson (ccODP) model, the stochastic model on which the chain ladder rests, with a feed-forward neural network. It follows Gabrielli, Richman and Wüthrich (2020), referred to as Paper C as in the code, and the restatement of Paper C by Härkönen (2021: 12-16). The ccODP model is fitted as a generalised linear model (GLM), and its maximum likelihood estimates (MLEs) become the fixed inputs of the network. The report covers the ccODP model, the neural network that is wrapped around it, the loss as Keras sees it, the gradients, and the properties of the starting point of gradient descent with the root mean square propagation (RMSprop) optimiser. The code that is explained is `fit_odp_glm()` in R/fit ODP GLM, `bccnn_model()` and `fit_bccnn()` in R/nn_models.R, the files R/loss functions.R and R/metrics.R, and the script `analysis/03_bCCNN/bCCNN fit.R`.

The report is the first of four companion explainers, referred to below as companions I to IV. Companion I is this document, *The mathematics behind the bCCNN*. Companion II, *The fitting procedure and training of the bCCNN*, describes which cells train the network, which cells stop the training and how the final network is chosen. Companion III, *The bCCNN as a whole: interpretation, evaluation and uncertainty*, joins the others and treats interpretation, evaluation and uncertainty. Companion IV, *The time-aware split for the bCCNN*, describes the rolling-origin split. The four documents share the notation summarised in the List of symbols (Appendix A).

The question addressed is what, exactly, the network computes and why it starts at the chain ladder: which model the network embeds, which criterion it minimises, how the gradient flows at the start and how far the first steps move the parameters, and how the reserves, the dispersion and the diagnostics follow from the fitted means. Each answer is stated as a formula, derived where it is not standard, and linked to the place in the code that computes it.

Three elements of the code are not in Paper C and are marked where they appear: the rolling-origin early stopping, which draws on Al-Mudafer, Avanzi, Taylor and Wong (2021) and is described in companions II and IV; the use of Keras's default initialiser for the hidden layers (Section 3.7, marked INFERRED in a comment of the script `bCCNN fit.R`); and the handling of accident or development periods without payments (Section 2.10), which is in neither Paper C nor Härkönen (2021).

1.2 THE CLAIMS TRIANGLE

The simulated portfolio has $n=20$ accident periods (years) $i=1, \dots, n$ and n development periods $j=1, \dots, n$. A cell (i, j) holds the incremental payments $Y_{i,j}$ made in development period j on claims of accident period i , that is, in calendar period

$$k=i+j-1. \quad (1.1)$$

At the valuation date, calendar period n , the observed cells are those with $k \leq n$ and the future cells those with $k > n$:

$$O=\{(i, j): i+j \leq n+1\}, F=\{(i, j): i+j > n+1\}, \\ |O|=n(n+1)/2=210, |F|=190. \quad (1.2)$$

The quantity to predict is the sum of the future cells, the outstanding payments to development period n ,

$$R = \sum_{(i,j) \in F} Y_{i,j}, R_i = \sum_{j > n+1-i} Y_{i,j}. \quad (1.3)$$

In the simulated data R is known: `triangle_sets()` returns it as the matrix test (the lower triangle, NA above the diagonal). Payments after development period n exist in the simulation but are in no triangle and no model; they are returned apart as `tail_o` and reported on their own row of the results table. The simulator's `triangle.csv` folds them into column n , which for accident period 1 is an observed cell; the script checks that folding and then uses the triangles of `triangle_sets()`, which stop at development period n .

In the code the square `sets$full` holds all $Y_{i,j}$, `sets$upper` holds O (future cells NA), and every amount is divided by $u=10^6$ before fitting (`dat_cfg$scale`), so that the fit runs in millions; Section 2.9 shows that this changes nothing but the intercept.

1.3 SCOPE AND LIMITATIONS

The report explains the mathematics of the model and its implementation. It does not report the reserves of the simulated portfolio, and it does not evaluate the bCCNN against the chain ladder; that is the subject of companion III. The fitting procedure around the model (the claims split, the rolling origin, early stopping and the final fit) is treated only where it touches the formulas, and is described in companions II and IV. Everything below assumes the configuration of the code: one 20×20 annual triangle, frozen embeddings and full-batch training.

Paper C is not quoted from the journal version. Its section, equation and listing numbers are those quoted in the comments of the code (its equations (3), (4), (5), (13) and (14), and its Listing 4), and its wording is quoted through the literature notes in the repository (thesis/Hyperparameter search), which cite the version on the Social Science Research Network (SSRN) (Gabrielli, Richman and Wüthrich, 2018) by PDF

page. Härkönen (2021) is cited by printed page and, where her formulas are used, by her own equation numbers; her Section 2.3.3 restates Paper C. Comparisons with the papers refer to their own data: Paper C's six simulated lines of business (LoBs) and Härkönen's simulated and real portfolios.

The numerical check of the formulas runs on a synthetic triangle and not on the simulated portfolio; it tests the formulas and says nothing about the data of the study. The check is a re-implementation in numpy and not a Keras run, so statements about the single-precision arithmetic of Keras are reasoned rather than measured.

1.4 VERIFICATION OF THE MATHEMATICS

Every formula of Chapters 2 to 5 was recomputed in plain numpy, without Keras, on a synthetic 20×20 triangle with the dimensions of the data of the study (210 observed cells, payments in millions), and the two implementations were compared. The results are in Section 6.3. In short: the iteratively weighted least squares (IWLS) fit of Section 2.6 reproduces the chain-ladder reserve to a relative error of 5×10^{-15} and satisfies the marginal-total equations to 10^{-11} ; the network at its starting point reproduces the ccODP means to 10^{-14} ; the analytic gradients of Chapter 4 agree with central finite differences to a relative error of 10^{-7} or better in every parameter group; the parameter count agrees with the 547 reported by Keras; and the predicted behaviour of the first RMSprop step (Section 4.6) is observed exactly.

1.5 OUTLINE OF THE DOCUMENT

Chapter 2 sets out the ccODP model: its assumptions and identification, the Poisson deviance, the maximum likelihood equations and their solution by IWLS, the proof that the model is the chain ladder, the dispersion, the unit of the payments and the treatment of periods without payments. Chapter 3 describes the network built around the ccODP model, its initialisation at the ccODP fit, and the loss that Keras minimises. Chapter 4 derives the gradients and the behaviour of the first gradient steps. Chapter 5 turns the fitted means into reserves, a dispersion and diagnostics. Chapter 6 maps every formula to the code, reports the numerical check and states the conclusions. The symbols are summarised in Appendix A.

CHAPTER 2

THE CROSS-CLASSIFIED OVER-DISPersed POISSON MODEL

2.1 INTRODUCTION

Chapter 1 defined the claims triangle and the quantity to predict. This chapter sets out the ccODP model from which the bCCNN starts: its assumptions and identification, the Poisson deviance by which it is fitted, the maximum likelihood equations and the way `glm()` solves them, the proof that the fitted model reproduces the chain-ladder reserve, the estimation of the dispersion, the effect of the unit of the payments and the treatment of periods without payments. The marginal-total equations derived here are used again in Chapter 4 to explain the gradient at the start of the training.

2.2 ASSUMPTIONS

The assumptions are those of Section 2 of Paper C and of Härkönen (2021: 5), her equations (1) to (3). The incremental payments of the observed cells are independent, with

$$\begin{aligned} E[Y_{i,j}] &= \mu_{i,j} = \exp\{c + \alpha_i + \beta_j\}, \\ \text{Var}(Y_{i,j}) &= \phi \mu_{i,j}, \phi > 0. \end{aligned} \tag{2.1}$$

The model is called 'cross-classified' because the two factors, accident period and development period, enter additively on the log scale: every accident period has the same development pattern $(e^{\beta_j})_j$, scaled by its own level $e^{c+\alpha_i}$. No interaction between i and j is allowed; that is exactly what the network of Chapter 3 adds.

The term 'over-dispersed Poisson' means that the variance is ϕ times the mean instead of equal to it. A concrete distribution with these moments is the scaled Poisson $Y = \phi P$ with $P \sim \text{Poisson}(\mu/\phi)$: then $E[Y] = \phi \cdot \mu/\phi = \mu$ and $\text{Var}(Y) = \phi^2 \cdot \mu/\phi = \phi \mu$. The fit, however, uses only the first two moments: it is a quasi-likelihood fit (family = `quasipoisson(link = "log")`), and ϕ is estimated afterwards (Section 2.8). The payments must be non-negative, and `fit_odp_glm()` stops if an observed increment is negative. Härkönen (2021: 28) replaces negative aggregated payments by zero; the simulated triangle has none.

2.3 IDENTIFICATION

The linear predictor $c + \alpha_i + \beta_j$ has $1 + n + n$ coefficients, but only $2n - 1$ of them are identifiable: adding a constant to all α_i and subtracting it from c changes nothing, and likewise for the β_j . Paper C's equation (3) fixes

$$\alpha_1 = \beta_1 = 0, \tag{2.2}$$

so that c is the log mean of cell $(1, 1)$, α_i is the log ratio of accident period i to accident period 1 at the same development period, and β_j is the log ratio of development period j to development period 1 within an accident period. In `fit_odp_glm()` this is the treatment contrasts of R for $y \sim \text{origin} + \text{dev}$, with origin and dev as factors whose first level is period 1: the intercept is c , the coefficient `origini` is α_i and `devj` is β_j . The number of parameters is

$$n_{par} = 1 + (n - 1) + (n - 1) = 2n - 1 = 39, \quad (2.3)$$

against $n_{obs} = |O| = 210$ fitted cells on the full triangle.

2.4 THE POISSON DEVIANCE

For a mean vector μ and a set of cells S , the (unscaled) Poisson deviance is Paper C's equation (4):

$$D(y, \mu; S) = 2 \sum_{(i,j) \in S} \left(\mu_{i,j} - y_{i,j} + y_{i,j} \log \frac{y_{i,j}}{\mu_{i,j}} \right), \quad (2.4)$$

$0 \log 0 := 0.$

It is `poisson_deviance(y, mu)` in R/loss functions.R, which drops the cells where y is NA; that is how the same function scores the observed triangle, a validation mask or the true lower triangle. Each summand is non-negative and zero only when $\mu_{i,j} = y_{i,j}$, because $x \mapsto \mu - x + x \log(x/\mu)$ is the Bregman divergence of $x \log x$.

Where it comes from. The Poisson log-likelihood of the observed cells, with the means scaled by ϕ as in the scaled-Poisson representation, is up to a constant

$$\ell(\mu) = \frac{1}{\phi} \sum_S (y_{i,j} \log \mu_{i,j} - \mu_{i,j}). \quad (2.5)$$

The saturated model sets $\mu_{i,j} = y_{i,j}$ and attains $\ell_{sat} = \frac{1}{\phi} \sum (y \log y - y)$. Twice the gap, times ϕ , is (2.4):

$$D = 2\phi (\ell_{sat} - \ell(\mu)). \quad (2.6)$$

Since ϕ only multiplies the whole log-likelihood, maximising (2.5) and minimising (2.4) give the same $\hat{\mu}$ for every ϕ : the dispersion plays no part in the point estimates, a remark that Härkönen (2021: 5) also makes. This is why the network can use the deviance as its loss and still estimate ϕ separately.

2.5 THE MAXIMUM LIKELIHOOD EQUATIONS

Write $a_{i,j} = \log \mu_{i,j} = c + \alpha_i + \beta_j$. Differentiating one summand of (2.4) in $a_{i,j}$ gives

$$\frac{\partial}{\partial a_{i,j}} 2(e^{a_{i,j}} - y_{i,j} + y_{i,j} \log y_{i,j} - y_{i,j} a_{i,j}) = 2(\mu_{i,j} - y_{i,j}). \quad (2.7)$$

By the chain rule, with $\partial a_{i,j} / \partial c = 1$, $\partial a_{i,j} / \partial \alpha_i = 1 \{i = i'\}$ and $\partial a_{i,j} / \partial \beta_j = 1 \{j = j'\}$,

$$\begin{aligned} \frac{\partial D}{\partial c} &= 2 \sum_{(i,j) \in O} (\mu_{i,j} - y_{i,j}), \\ \frac{\partial D}{\partial \alpha_i} &= 2 \sum_{j \leq n+1-i} (\mu_{i,j} - y_{i,j}), \\ \frac{\partial D}{\partial \beta_j} &= 2 \sum_{i \leq n+1-j} (\mu_{i,j} - y_{i,j}). \end{aligned} \quad (2.8)$$

At the MLE all of these vanish: for $i, j \geq 2$ directly, and for $i=1$ and $j=1$ because the total minus the others vanishes too. The fitted ccODP therefore reproduces every observed row total and every observed column total of the triangle:

$$\begin{aligned} \sum_{j \leq n+1-i} \hat{\mu}_{i,j}^{cc} &= \sum_{j \leq n+1-i} y_{i,j} (i=1, \dots, n), \\ \sum_{i \leq n+1-j} \hat{\mu}_{i,j}^{cc} &= \sum_{i \leq n+1-j} y_{i,j} (j=1, \dots, n). \end{aligned} \quad (2.9)$$

This 'marginal totals' property is the whole reason why the ccODP is the chain ladder (Section 2.7), and it is also why two of the gradients of the network vanish at the start (Section 4.5). The numerical check finds the row and column sums of $\hat{\mu} - y$ at 10^{-11} .

2.6 HOW GLM() SOLVES THEM: IWLS

`stats::glm()` solves (2.8) by Fisher scoring, which for a GLM is IWLS (McCullagh and Nelder, 1989). With the design matrix X (210×39 : a column of ones, 19 accident-period dummies and 19 development-period dummies) and the coefficient vector $\vartheta = (c, \alpha_2, \dots, \alpha_n, \beta_2, \dots, \beta_n)$, each iteration computes, from the current ϑ ,

$$\begin{aligned} \eta &= X \vartheta, \mu = e^\eta, z = \eta + \frac{y - \mu}{\mu} \text{ (working response)}, \\ W &= \text{diag}(\mu) \text{ (working weights)}, \\ \vartheta_{new} &= (X^T W X)^{-1} X^T W z. \end{aligned} \quad (2.10)$$

For the log link the working weight μ comes from $(\partial \mu / \partial \eta)^2 / \text{Var} = \mu^2 / \mu$. The code runs the iteration with `glm.control(epsilon = 1e-12, maxit = 100)`, so the relative change of the deviance between iterations has to fall below 10^{-12} ; it warns if that is not reached and stops if a coefficient is NA (the fitted cells then do not connect all periods). The same routine fits a subset of the observed cells when a cells mask is given (rolling-origin partitions); then (2.8) and (2.9) hold over the fitted cells only.

2.7 THE CCODP IS THE CHAIN LADDER

Statement. Let $\hat{\mu}^{cc}$ be the ccODP MLE on the full observed triangle, and define the fitted and predicted cumulative payments $\hat{C}_{i,j} = \sum_{l \leq j} \hat{\mu}_{i,l}^{cc}$. Then $\hat{C}_{i,n}$ equals the chain-ladder ultimate of accident period i , that is, the latest cumulative payment $C_{i,n+1-i} = \sum_{j \leq n+1-i} y_{i,j}$ multiplied by the volume-weighted chain-ladder factors,

$$\begin{aligned} \hat{C}_{i,n} &= C_{i,n+1-i} \prod_{k=n+1-i}^{n-1} f_k, \\ f_k &= \frac{\sum_{i' \leq n-k} C_{i',k+1}}{\sum_{i' \leq n-k} C_{i',k}}, \end{aligned} \quad (2.11)$$

so that the ccODP reserve $\hat{R} = \sum_F \hat{\mu}_{i,j}^{cc}$ is the chain-ladder reserve (Hachemeister and Stanard, 1975; Mack, 1991; Renshaw and Verrall, 1998; Gabrielli *et al.*, 2020). Renshaw and Verrall (1998) is also the general reference of Härkönen (2021: 5) for this model, and Section 2 of Paper C states the result. The script asserts it: `all.equal(res$odp$reserve * scale, cl$total[["ibnr"]])`.

Proof. Write $\hat{\mu}_{i,j}^{cc} = \hat{a}_i \hat{b}_j$ with $\hat{a}_i = e^{\hat{c} + \hat{\alpha}_i}$ and $\hat{b}_j = e^{\hat{\beta}_j}$, and $S_k = \sum_{j \leq k} \hat{b}_j$, so that $\hat{C}_{i,k} = \hat{a}_i S_k$. Fix $k \in \{2, \dots, n\}$.

Step 1. Sum the column equations of (2.9) over the columns $1, \dots, k$. The cells involved are the observed cells in those columns. They split into the rows $i \leq n+1-k$, for which all cells $(i, 1), \dots, (i, k)$ are observed, and the rows $i \geq n+2-k$, whose observed cells all lie in columns $\leq n+1-i \leq k-1$, so that the whole observed row is included. For those later rows the row equations of (2.9) state that the fitted row total equals the observed row total; subtracting them leaves

$$\sum_{i \leq n+1-k} \hat{C}_{i,k} = \sum_{i \leq n+1-k} C_{i,k}. \quad (2.12)$$

Step 2. Apply (2.12) with $k-1$ in place of k : it runs over the rows $i \leq n+2-k$. The extra row $i = n+2-k$ has observed cells exactly up to column $k-1$, so its row equation reads $\hat{C}_{n+2-k,k-1} = C_{n+2-k,k-1}$. Subtracting it gives

$$\sum_{i \leq n+1-k} \hat{C}_{i,k-1} = \sum_{i \leq n+1-k} C_{i,k-1}. \quad (2.13)$$

Step 3. Divide (2.12) by (2.13): the right-hand side is the chain-ladder factor f_{k-1} of (2.11); the left-hand side is $\hat{a}_i S_k / (\hat{a}_i S_{k-1})$ with $\hat{a}_i = \sum_{i \leq n+1-k} \hat{a}_i$. Hence

$$\frac{S_k}{S_{k-1}} = f_{k-1}, k=2, \dots, n. \quad (2.14)$$

Step 4. The row equation of (2.9) for accident period i states that $\hat{a}_i S_{n+1-i} = C_{i,n+1-i}$. Therefore

$$\hat{C}_{i,n} = \hat{a}_i S_n = C_{i,n+1-i} S_n / S_{n+1-i} = C_{i,n+1-i} \prod_{k=n+2-i}^n S_k / S_{k-1}, \text{ which by (2.14) is (2.11). } \square$$

Two consequences are used later. First, (2.14) states that the fitted development pattern of the ccODP is the chain-ladder pattern, the same for every accident period (Section 5.4). Second, the proof uses only the marginal-total equations (2.9); the bCCNN does not satisfy them after the first gradient step, so its reserve is no longer a chain-ladder reserve.

2.8 DISPERSION

The point estimates do not need ϕ , but the variance (2.1), the Pearson residuals and any standard error do. `fit_odp_glm()` returns both usual estimators, with the degrees of freedom $n_{obs} - n_{par} = 210 - 39 = 171$ on the full triangle:

$$\begin{aligned} \hat{\phi}_D &= \frac{D(y, \hat{\mu}^{cc}; O)}{n_{obs} - n_{par}}, \\ \hat{\phi}_P &= \frac{1}{n_{obs} - n_{par}} \sum_O \frac{(y_{i,j} - \hat{\mu}_{i,j}^{cc})^2}{\hat{\mu}_{i,j}^{cc}}. \end{aligned} \quad (2.15)$$

The deviance estimator $\hat{\phi}_D$ is Paper C's equation (5) and the default of the code. Both are moment estimators of the ratio of variance to mean: under (2.1) each Pearson term has expectation about ϕ , and the deviance term agrees with it to second order. `config.yml` selects `odp: phi: deviance`. The dispersion of the bCCNN is a separate matter (Section 5.3).

2.9 THE UNIT OF THE PAYMENTS

The fit runs on y/u with $u = 10^6$. Let μ' be the MLE on $y' = y/u$. Since $\log(\mu_{i,j}/u) = (c - \log u) + \alpha_i + \beta_j$, the scaled model is the same model with the intercept shifted, and the deviance scales linearly:

$$\begin{aligned} D(y/u, \mu/u) &= \frac{1}{u} D(y, \mu), \text{ so} \\ \hat{c}' &= \hat{c} - \log u, \hat{\alpha}' = \hat{\alpha}, \hat{\beta}' = \hat{\beta}, \\ \hat{\mu}' &= \hat{\mu}/u, \hat{R}' = \hat{R}/u, \hat{\phi}' = \hat{\phi}/u. \end{aligned} \quad (2.16)$$

The Pearson residual $(y - \mu)/\sqrt{\phi\mu}$ is invariant (u cancels), and so is the relative difference $\mu^{bCCNN}/\mu^{cc} - 1$. The numerical check confirms (2.16) to 10^{-13} . For the network the unit matters only through the fixed inputs: α_i and β_j do not change with u , so the hidden layers see the same inputs

whatever the unit, and the output intercept starts at $\hat{c}'$. Unlike the steps of stochastic gradient descent (SGD), the steps of RMSprop are (almost) invariant to the scale of the gradient (companion II, Section 3.3), so the choice of unit changes the training very little.

2.10 PERIODS WITHOUT PAYMENTS

2.10.1 What happens

If an accident period or a development period has no payments in the fitted cells, its MLE is $-\infty$. In the row or column equation (2.9) the fitted means of that period must sum to zero, which only $e^{-\infty}=0$ achieves, so the deviance keeps decreasing as the effect goes to $-\infty$. `glm()` stops at a large negative value (about -30) that it reports as converged, with fitted means of order e^{-30} . `fit_odp_glm()` flags these periods as `zero_origin` and `zero_dev` and warns.

2.10.2 Why it is harmless for the chain ladder

A development period with no payments has a chain-ladder factor of exactly 1, and the ccODP limit $\mu=0$ for that column predicts the same. The difficulty is purely numerical, and it matters in two places.

As a network input. The embedding of that period is then ≈ -30 , far outside the range of the other effects (companion II, Section 3.4, on the raw inputs). The hidden layers see a value they were never trained near.

In the deviance. A cell with $y>0$ scored against $\mu \approx e^{-30}$ adds $y \log(y/\mu) \approx 30 y$ to the deviance. One such cell can dominate a validation or test loss: in the check of the code change, a single validation cell raised the starting validation deviance of a partition from 32 to 3 047. `poisson_deviance()` does not catch it, because e^{-30} is positive and finite.

2.10.3 What Paper C and Härkönen do

Neither discusses it. Härkönen replaces negative aggregated payments with zero so that the Poisson loss applies (Härkönen, 2021: 28, 45; her Sections 3.1 and 4.1), which makes every cell valid but does nothing about a period whose payments sum to zero. When she removes the last diagonal she notes that the networks "require input values for each accident year, reporting delay and payment delay" and declares them "not applicable" (Härkönen, 2021: 37), instead of handling the gap. Her triangles have 12 accident years and portfolios of about 100 000 to over a million claims (Härkönen, 2021: 27, 45), and Paper C's six simulated LoBs are of the same kind, so every period has payments in both halves of their balanced claims splits. Paper C's Listing 4 sets the embeddings to the GLM estimates with no guard. Paper C is cited here through the quotations in the literature notes in the repository (thesis/Hyperparameter search).

2.10.4 What the code does

The helper `mask_zero_periods(m, fit)` in `R/nn_models.R` sets to NA the cells of `m` in the periods that fit flags, so that they are left out of the deviance:

- Under the claims split, `bccnn_validation()` masks the validation cells in periods with no payments in the training half (companion II, Section 2.3.3);
- Under the rolling origin, `bccnn_ro_partition()` masks the validation cells in periods with no payments in the training cells of the partition, and the test cells in periods with no payments in *any* cell observed at the valuation date (companion IV, Sections 3.3 and 3.4); `n_vali` and `n_test` count the cells actually scored.

The network input itself is left as it is. On the full annual triangle every period has payments; whether a training half or a rolling-origin partition has an empty period depends on how sparse the last development years are.

2.11 SUMMARY

The ccODP model describes each cell by an intercept, an accident-period effect and a development-period effect on the log scale, with a variance proportional to the mean. It is fitted by minimising the Poisson deviance, which gives the same point estimates for every dispersion, and `glm()` finds the minimum by IWLS. The maximum likelihood equations state that the fitted model reproduces every observed row and column total, and these marginal totals alone imply that the ccODP reserve is the chain-ladder reserve and that its development pattern is the chain-ladder pattern. The dispersion is estimated afterwards from the deviance or the Pearson statistic, the unit of the payments shifts only the intercept, and periods without payments produce effects of about -30 that the code masks out of the deviance but leaves in the network input. Chapter 3 wraps a neural network around this model and starts it exactly at the ccODP fit.

CHAPTER 3

THE bccnn NETWORK AND ITS LOSS

3.1 INTRODUCTION

Chapter 2 established that the fitted ccODP is the chain ladder. This chapter describes the network that the bccnn wraps around it, following Section 3 and Listing 4 of Paper C and Härkönen (2021: 12-16): the embedding layers that hold the ccODP effects, the skip connection that carries the ccODP linear predictor to the output, the feed-forward part, the output neuron, dropout and the initialisation at the ccODP fit. It closes with the loss that Keras minimises and with what the code records after each gradient step.

Figure 3.1 draws the network with the layer names of `bccnn_model()`. The green path is the skip connection, which carries the ccODP linear predictor unchanged to the output; the orange path is the feed-forward network; the output neuron adds the two on the log scale and exponentiates.

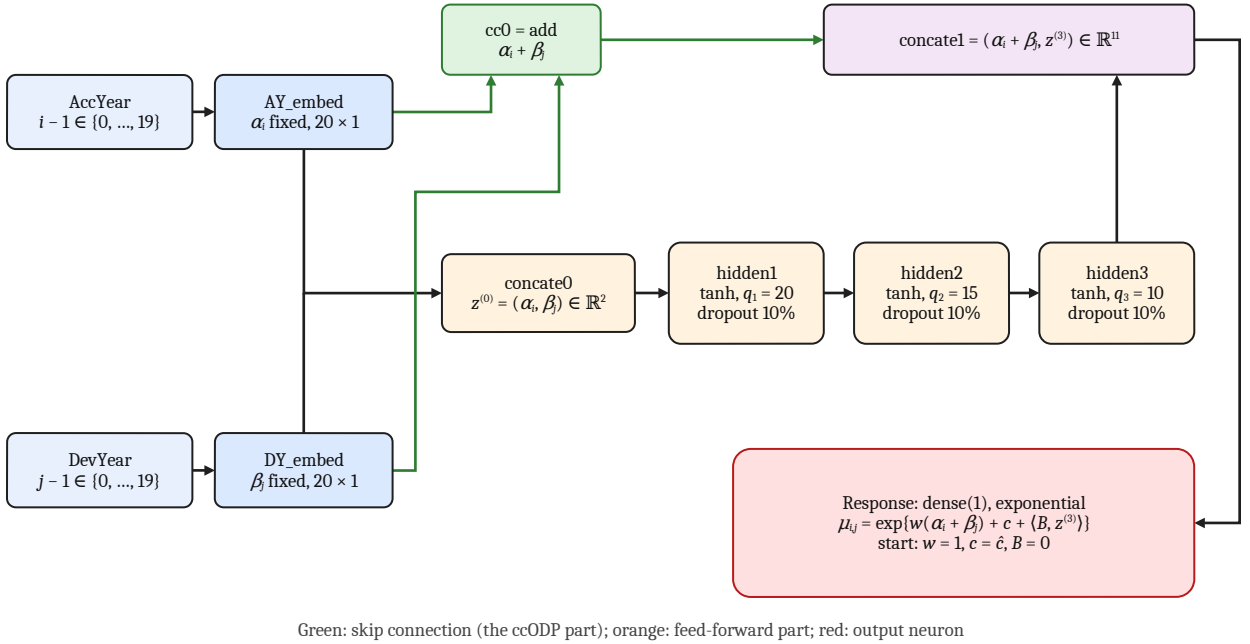


Figure 3.1 The network of `bccnn_model()` with the Keras layer names

3.2 EMBEDDING LAYERS

The inputs are the categorical accident period and development period of a cell, passed to Keras as the 0-based integers $i - 1$ and $j - 1$ (`bccnn_inputs()`). An embedding layer maps a category to a vector; here each vector is one-dimensional and holds the ccODP effect of that period (Paper C's equations (9) and (10); Härkönen, 2021: 13, her equations (33) and (34)):

$$\text{AY_embed} : i \mapsto \hat{\alpha}_i, \text{DY_embed} : j \mapsto \hat{\beta}_j. \quad (3.1)$$

The two embedding matrices (20×1 each, 40 numbers) are set to the MLE with `set_weights()` and frozen (`trainable = trainable_embeddings, false` in `config.yml`), as Härkönen does and as Paper C's Listing 4 does. If they were trainable, 40 parameters would be added to θ and the skip connection would stop being exactly the ccODP after the first step; Härkönen (2021: 33-34) reports that the predictions then improved for some LoBs at a higher computation cost (her rows labelled TW, for trainable input weights). Everything below assumes that the embeddings are fixed.

3.3 THE SKIP CONNECTION

`cc0 = layer_add(AY_flat, DY_flat)` is the ccODP linear predictor without its intercept,

$$cc_0(i, j) = \hat{\alpha}_i + \hat{\beta}_j. \quad (3.2)$$

It goes straight to the output neuron (the green path of Figure 3.1). This is Härkönen's 'skip-connection' (Härkönen, 2021: 12, 18) and Paper C's 'embedding' of the ccODP: the classical model is a sub-network of the bCCNN.

3.4 THE FEED-FORWARD PART

The same two numbers are also concatenated into the input of a feed-forward network,

$$z^{(0)}(i, j) = (\hat{\alpha}_i, \hat{\beta}_j)^T \in R^2, \quad (3.3)$$

followed by three dense layers with hyperbolic tangent activation and $q_1=20$, $q_2=15$ and $q_3=10$ neurons (Härkönen, 2021: 14, her equation (40); Paper C, Listing 4):

$$z^{(h)}(i, j) = \tanh(b^{(h)} + W^{(h)T} z^{(h-1)}(i, j)) \in R^{q_h}, h=1, 2, 3, \quad (3.4)$$

where $\tanh$ acts component-wise, $W^{(h)} \in R^{q_{h-1} \times q_h}$ and $b^{(h)} \in R^{q_h}$. A dropout layer with rate $p=0.1$ follows each hidden layer (Section 3.6). Härkönen (2021: 15) gives two reasons for $\tanh$: its derivative is $1 - \tanh^2$, cheap to compute from the activation itself (Section 4.4), and its range $(-1, 1)$ keeps the activations bounded.

The trainable parameters are counted in Table 3.1, which follows Härkönen's (2021: 23) count, her equation (76), with $q_0=2$ and $q_4=1$, plus the skip weight. The total of 547 agrees with the `summary()` of the model in Keras, where the 40 embedding numbers appear as non-trainable.

Table 3.1 Parameters of the network

Layer	Input $\times$ output	Weights	Biases	Trainable parameters
hidden1	2×20	40	20	60
hidden2	20×15	300	15	315
hidden3	15×10	150	10	160
Response	11×1	11	1	12
Total trainable				547
Embeddings (frozen, 40 non-trainable)	20×1 , twice	40	0	0

3.5 THE OUTPUT NEURON

concatel joins the skip connection and the last hidden layer into an 11-vector $(c c_0(i, j), z^{[3]}(i, j)) \in R^{11}$, and Response is a dense layer with one neuron and exponential activation. Writing its 11 weights as (w, B) , with $w \in R$ on the skip connection and $B \in R^{10}$ on the hidden features, and its bias as c , the mean of the network for cell (i, j) is Paper C's equation (13):

$$\mu_{i,j}(\theta) = \exp \left\{ \underbrace{w(\hat{\alpha}_i + \hat{\beta}_j)}_{\text{skip connection}} + \underbrace{c + \text{ } \text{ } \text{ } B, z^{[3]}(i, j)}_{\text{network}} \right\}, \quad (3.5)$$

$$a_{i,j} := \log \mu_{i,j}.$$

Paper C keeps w trainable, as `bccnn_model()` does; the comment in the script `bCCNN fit.R` describes this as "exactly as Listing 4 does". Härkönen (2021: 15), her equation (44), writes the skip connection with weight 1. Both views of (3.5) are useful.

Multiplicative boosting. Factoring out the ccODP mean gives $\mu_{i,j} = \hat{\mu}_{i,j}^{cc} \cdot \exp \{ (w - 1)(\hat{\alpha}_i + \hat{\beta}_j) + (c - \hat{c}) + \text{ } \text{ } \text{ } B, z^{[3]}(i, j) \}$. The network learns a multiplicative correction of the chain-ladder means, which is 1 at the start. This is the combined actuarial neural network (CANN) idea of Wüthrich and Merz (2019), on which Härkönen (2021: 4) also draws, and what Paper C calls "neural network boosting" (Gabielli *et al.*, 2018: 14).

A function of two scalars. The feed-forward part sees only $(\hat{\alpha}_i, \hat{\beta}_j)$. Whatever it learns is a smooth surface $g(\alpha, \beta)$ evaluated at the 20 accident effects and the 20 development effects. Two accident periods with equal $\hat{\alpha}$ receive the same correction at every development period; the correction for a future cell (i, j) is the surface at the pair $(\hat{\alpha}_i, \hat{\beta}_j)$, a combination never seen in training even though each coordinate was. What the network can add to the ccODP is therefore exactly an interaction between the accident-period effect and the development-period effect, that is, accident-period-dependent development patterns (Section 5.4, cumulative factors).

3.6 DROPOUT

The dropout of Keras is 'inverted dropout' (Srivastava, Hinton, Krizhevsky, Sutskever and Salakhutdinov, 2014; Härkönen, 2021: 12, her equations (31) and (32)). During training, after each hidden layer a mask of independent Bernoulli variables is drawn and the surviving activations are scaled up so that their expectation is unchanged:

$$\tilde{z}^{(h)} = \frac{z^{(h)} \odot m^{(h)}}{1-p}, m_r^{(h)} \sim \text{Bernoulli}(1-p) \text{ independently,} \quad (3.6)$$

$$E[\tilde{z}^{(h)} / z^{(h)}] = z^{(h)}.$$

The next layer reads $\tilde{z}^{(h)}$ in place of $z^{(h)}$ in (3.4), and the output neuron reads $\tilde{z}^{(3)}$ in (3.5). At prediction time (predict_on_batch, predict) no mask is drawn and (3.4) and (3.5) hold as written. This has two consequences for what the code records (Section 3.8.2): the loss that Keras prints during training is the loss of a randomly thinned network, and it is reported as the column train_dropout; the deviances in the columns train, vali, test and truth are computed from predict_on_batch() after each step, without dropout. Paper C's Figure 2 shows the first kind of curve (noisy), and Paper C's Table 3 reports both kinds. Härkönen's (2021: 12) remark on dropout, that each neuron must learn features that are useful without relying on particular other neurons, is the reason why it regularises a network with 547 parameters on 210 cells.

3.7 THE INITIALISATION

Keras builds the model with its default initialisers: Glorot-uniform weights, $W_{r,s}^{(h)} \sim U(-\lambda_h, \lambda_h)$ with $\lambda_h = \sqrt{6/(q_{h-1} + q_h)}$, and zero biases, for the three hidden layers and for Response; set_random_seed(seed) fixes the draw. Then bccnn_model() overwrites the embeddings and the output neuron, which gives the start of gradient descent of Paper C's equation (14):

$$\begin{aligned} \theta_0: \quad \text{AY_embed} &= \hat{\alpha}, \text{DY_embed} = \hat{\beta}, \\ w &= 1, c = \hat{c}, B = 0 \in R^{10}, \\ (W^{(h)}, b^{(h)}) &\text{random: Glorot-uniform weights, zero biases, } h = 1, 2, 3. \end{aligned} \quad (3.7)$$

Property 1: the start is the ccODP model. With $B=0$ the hidden layers do not reach the output, whatever their random weights, and (3.5) becomes $\mu_{i,j}(\theta_0) = \exp\{\hat{\alpha}_i + \hat{\beta}_j + \hat{c}\} = \hat{\mu}_{i,j}^{cc}$ for every cell, observed or future. Hence the deviance at step 0 equals the ccODP deviance, the reserve at step 0 equals the chain-ladder reserve (Section 2.7), and every cumulative development factor at step 0 is the chain-ladder factor. The record at epoch = 0 of the history is this start. Keras holds the weights in single precision, so in R the equality holds to about 10^{-7} relative; in the double-precision check it holds to 10^{-14} .

This is what makes the bCCNN a calibration of the chain ladder rather than a new model: the number of gradient steps measures how far the network is allowed to move away from the chain ladder, and $t^* = 0$ returns the chain ladder itself.

The initialiser of the hidden layers. A comment marked INFERRED in the script bCCNN fit.R records that the Python version of the single bCCNN used the default initialiser of torch, $U(\pm 1/\sqrt{\text{fan-in}})$, for the hidden layers, and observes that it leaves the chain ladder about three times more slowly. The reason is visible in Section 4.5: the only non-zero gradient at the start is the one in B , and it is proportional to the hidden activations $z^{(3)}$; smaller initial weights make $z^{(3)}$ smaller and closer to zero, so the first steps add less. Paper C's own default is that of Keras, which the R code keeps.

3.8 THE LOSS AS KERAS SEES IT

3.8.1 Keras's Poisson loss and the deviance

`compile(loss = "poisson")` minimises the mean over the N training cells of $\mu - y \log(\mu + \varepsilon)$, with the small constant $\varepsilon = 10^{-7}$ of Keras inside the logarithm,

$$\ell(\theta) = \frac{1}{N} \sum_{r=1}^N (\mu_r(\theta) - y_r \log(\mu_r(\theta) + \varepsilon)). \quad (3.8)$$

Dropping ε (its effect on ℓ is below 10^{-7} here), (3.8) and the deviance (2.4) over the same cells differ by a constant that does not depend on θ :

$$D(y, \mu(\theta); S) = 2N \ell(\theta) + 2 \sum_{r=1}^N (y_r \log y_r - y_r). \quad (3.9)$$

Minimising the loss of Keras is therefore minimising the Poisson deviance, the same criterion by which the ccODP was fitted (Paper C, Section 3.3; Härkönen, 2021: 21, her equation (72)), and the gradients differ only by the factor $2N$. The function `keras_poisson_to_deviance(loss, y)` is (3.9); the code applies it to the loss that Keras reports after each epoch to obtain the column `train_dropout` in the units of the other columns.

3.8.2 What is recorded after every step

`fit_bccnn()` keeps every cell of the $n \times n$ square in `x_all` and the training cells in `x_fit`. After each gradient step a Keras callback runs `predict_on_batch(x_all)` (without dropout), reshapes the result to the $n \times n$ matrix $\mu^{(t)}$ and stores one row of the history, as set out in Table 3.2.

Table 3.2 The history recorded by `fit_bccnn()`

Column	Formula	Cells
epoch	Step t (0 = the ccODP start)	
train_dropout	$2 N \ell_t^{drop} + 2 \sum (y \log y - y)$, from the loss Keras reported for epoch t (with dropout; computed before the update of that epoch, so it belongs to θ_{t-1})	Training cells
train	$D(y, \mu^{(t)}; \text{training cells})$	Training cells
vali, test, truth	$D(\cdot, \mu^{(t)}; \cdot)$ on the cells of each tracked matrix (NA elsewhere)	Validation, test, true lower triangle
*_pred	$\sum \mu_{i,j}^{(t)}$ over the cells of each tracked matrix	The same

With `batch_size = NULL` the batch is all N training cells, so one Keras epoch is exactly one gradient-descent step and epoch counts steps, as Paper C does when it asks "how many iterations of the gradient descent algorithm we should perform" (Gabrielli *et al.*, 2018: 11). Companion II (Section 4.2) uses these columns for the stopping rule.

3.9 SUMMARY

The bCCNN feeds the ccODP effects of a cell, held in two frozen one-dimensional embeddings, both straight to the output through a skip connection and through three hidden layers of 20, 15 and 10 hyperbolic tangent neurons with dropout. The output neuron adds the two paths on the log scale, so the network is the chain ladder multiplied by a correction that depends only on the pair of effects, which makes it an interaction between accident and development periods. With the output weights of the hidden features set to zero, the skip weight to one and the intercept to the ccODP intercept, the network starts exactly at the ccODP means, whatever the random hidden weights. Keras's Poisson loss equals the Poisson deviance up to a constant and the factor $2 N$, and the code records the deviances without dropout after every step. Chapter 4 derives the gradients of this loss and shows what happens at the first steps from the ccODP start.

CHAPTER 4

GRADIENTS AND THE START OF GRADIENT DESCENT

4.1 INTRODUCTION

Chapter 3 defined the network, its start θ_0 and the loss (3.8). This chapter writes out the gradient that Keras computes by automatic differentiation for the loss (3.8); for the deviance it is multiplied by $2N$. Everything is per cell $r = (i, j)$ of the training set, summed over the batch. The chapter then combines the gradient with the marginal-total equations of Chapter 2 to show which parameters move at the start, and with the RMSprop update to show how far they move.

4.2 DERIVATIVE IN THE OUTPUT

From (2.7) and (3.8), with $a_r = \log \mu_r$,

$$\delta_r := \frac{\partial \ell}{\partial a_r} = \frac{1}{N} (\mu_r - y_r). \quad (4.1)$$

The signal that drives every parameter is the residual $\mu_r - y_r$ of the cell: positive where the network over-predicts, negative where it under-predicts, and zero in a cell that is fitted exactly.

4.3 OUTPUT LAYER

From (3.5), $\partial a_r / \partial c = 1$, $\partial a_r / \partial w = \hat{\alpha}_i + \hat{\beta}_j$ and $\partial a_r / \partial B = \tilde{z}_r^{(3)}$, so that

$$\begin{aligned} \frac{\partial \ell}{\partial c} &= \sum_r \delta_r, \quad \frac{\partial \ell}{\partial w} = \sum_r \delta_r (\hat{\alpha}_i + \hat{\beta}_j), \\ \frac{\partial \ell}{\partial B} &= \sum_r \delta_r \tilde{z}_r^{(3)} \in R^{10}. \end{aligned} \quad (4.2)$$

4.4 HIDDEN LAYERS (BACK-PROPAGATION)

Let $\gamma_r^{(h)} \in R^{q_h}$ be the derivative of ℓ in the pre-activation $u_r^{(h)} = b^{(h)} + W^{(h)} \tau \tilde{z}_r^{(h-1)}$ of layer h . Since $z = \tanh(u)$ has $\partial z / \partial u = 1 - z^2$, and the mask of (3.6) multiplies the activation, back-propagation (Goodfellow, Bengio and Courville, 2016) gives

$$\begin{aligned} \gamma_r^{(3)} &= \delta_r \left(B \odot \frac{m_r^{(3)}}{1-p} \right) \odot (1 - z_r^{(3)2}), \\ \gamma_r^{(h)} &= \left(W^{(h+1)} \gamma_r^{(h+1)} \odot \frac{m_r^{(h)}}{1-p} \right) \odot (1 - z_r^{(h)2}), \quad h=2, 1, \end{aligned} \quad (4.3)$$

and the parameter gradients are

$$\frac{\partial \ell}{\partial W^{(h)}} = \sum_r \tilde{z}_r^{(h-1)} y_r^{(h)T} \in R^{q_{h-1} \times q_h}, \frac{\partial \ell}{\partial b^{(h)}} = \sum_r y_r^{(h)}, \quad (4.4)$$

$$h=1, 2, 3, \tilde{z}_r^{(0)} = z_r^{(0)}.$$

Without dropout every mask is set to $1 - p$, so that the factors $m/(1 - p)$ become 1. The numerical check compares (4.2) to (4.4) (without dropout) with central finite differences of the deviance at the start and at a random point: the largest relative error over all nine parameter groups is 1.1×10^{-6} (in $W^{(3)}$), and the others are at 10^{-7} or below (Section 6.3).

4.5 PROPERTY 2: THE GRADIENT AT THE START

At θ_0 the means are the ccODP means, so $\delta_r = (\hat{\mu}_r^{cc} - y_r)/N$, and the marginal-total equations (2.9) give the following:

- The intercept gradient $\partial \ell / \partial c = \frac{1}{N} \sum_o (\hat{\mu}^{cc} - y)$ is zero (the total);
- The skip-weight gradient $\partial \ell / \partial w = \frac{1}{N} \left(\sum_i \hat{\alpha}_i \sum_j (\hat{\mu}_{i,j}^{cc} - y_{i,j}) + \sum_j \hat{\beta}_j \sum_i (\hat{\mu}_{i,j}^{cc} - y_{i,j}) \right)$ is zero (the row and column totals);
- Every hidden-layer gradient is zero, because $B=0$ makes $y_r^{(3)}=0$ in (4.3) and hence $y^{(2)}=y^{(1)}=0$;
- The gradient $\partial \ell / \partial B = \frac{1}{N} \sum_r (\hat{\mu}_r^{cc} - y_r) \tilde{z}_r^{(3)}$ is the only non-zero component.

The first gradient step therefore changes nothing but B : it adds to the ccODP log-mean the linear combination $\langle \Delta B, z^{(3)}(i, j) \rangle$ of the ten random hidden features that is most correlated (in the sense of (4.2)) with the chain-ladder residuals. This is a boosting step on the residuals of the chain ladder with random features, nothing else. From the second step on, $B \neq 0$ and the gradient flows back into the hidden layers, which then start to shape the features themselves; c and w also start to move, because the marginal-total equations no longer hold once $B \neq 0$.

Figure 4.1 shows the norms of the nine gradient groups of the loss of Keras at the start and after one and two full-batch RMSprop steps in the numerical check, without dropout; bars missing at step 0 are exactly zero or at machine precision. At step 0 only B is non-zero (c and w are at 10^{-14} , the hidden layers at exactly 0); after one step all are non-zero, with c , w and B two orders of magnitude above the hidden layers.

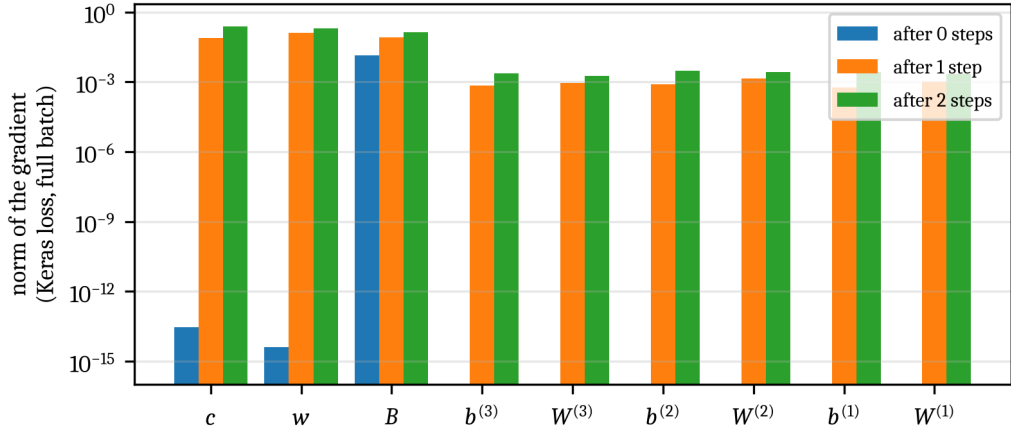


Figure 4.1 Norms of the gradient by parameter group over the first RMSprop steps in the numerical check

This property holds on the full triangle and on the training half of the claims split (the ccODP is refitted on the training cells, so (2.9) holds there). It holds equally for a rolling-origin partition, where the ccODP is fitted on the training cells of the partition by the same `glm()` call: (2.9) then holds over those cells.

4.6 PROPERTY 3: THE SIZE OF THE FIRST STEPS

The optimiser is RMSprop with the parameterisation of Keras (companion II, Section 3.3; Goodfellow *et al.*, 2016): for each coordinate ϑ of θ with gradient g_t ,

$$\begin{aligned} v_t &= \rho v_{t-1} + (1 - \rho) g_t^2, v_0 = 0, \\ \vartheta_t &= \vartheta_{t-1} - \eta \frac{g_t}{\sqrt{v_t} + \epsilon}. \end{aligned} \quad (4.5)$$

At the first step $v_1 = (1 - \rho) g_1^2$, so the update is $\eta g_1 / (\sqrt{1 - \rho} |g_1| + \epsilon)$. For every coordinate whose gradient is not tiny ($|g_1| \gg \epsilon / \sqrt{1 - \rho} \approx 3 \times 10^{-7}$) this is

$$\Delta \vartheta_1 \approx - \frac{\eta}{\sqrt{1 - \rho}} \text{sign}(g_1) = \mp 0.00316, \quad (4.6)$$

independent of the size of the gradient. Hence the first step moves each of the ten coordinates of B by ± 0.00316 exactly, and nothing else (the coordinates c and w , whose gradient is 10^{-14} , move by $\eta g / \epsilon \approx 10^{-10}$). The check reports $\max |\Delta B_1| = 0.0031621$ against 0.0031623 from (4.6).

Two things follow. First, the early path away from the chain ladder is set by η , ρ and the random features, not by how badly the chain ladder fits: a triangle that the chain ladder fits well and one that it fits badly both move by the same amount at step 1. Second, because the step is sign-like rather than proportional, the training deviance need not fall at first: in the check it goes $108.967 \rightarrow 109.034 \rightarrow 109.198 \rightarrow 108.973$ over the first three steps before it starts to decrease. The loss curves of Paper C and of the code in the repository start with the same kind of wobble.

4.7 SUMMARY

The gradient of every parameter is driven by the cell residuals $\mu_r - y_r$, passed through the output neuron and, by back-propagation, through the hidden layers. At the ccODP start the marginal-total equations make the gradients in the intercept and the skip weight vanish, and the zero output weights B block every hidden-layer gradient, so the first step changes only B : it is a boosting step on the chain-ladder residuals with ten random features. Because the first RMSprop step is sign-like, each coordinate of B moves by $\eta/\sqrt{1-\rho} \approx 0.00316$ whatever the size of the gradient, and the training deviance may rise for a few steps before it falls. Chapter 5 turns the fitted means at the chosen step into reserves, a dispersion and diagnostics.

CHAPTER 5

RESERVES, DISPERSION AND DIAGNOSTICS

5.1 INTRODUCTION

Chapters 2 to 4 described how the means of the ccODP and of the bCCNN are obtained. This chapter shows what is computed from those means: the reserves by accident period and in total, the dispersion of the bCCNN, for which the GLM estimator of Chapter 2 does not exist, and three diagnostics that compare the network with the chain ladder.

5.2 FROM THE FITTED MEANS TO THE RESERVES

Whatever the model, the reserve is the sum of the predicted means over the future cells (Paper C, Section 3.3.3):

$$\hat{R}_i = \sum_{j>n+1-i} \mu_{i,j}, \hat{R} = \sum_{i=1}^n \hat{R}_i = \sum_{(i,j) \in F} \mu_{i,j}. \quad (5.1)$$

`reserve_by_origin(y, mu)` returns, per accident period, the latest cumulative payment $C_{i,n+1-i}$ (latest), $\hat{R}_i$ (ibnr), their sum (ultimate) and the ratio `dev_to_date` $= C_{i,n+1-i} / (C_{i,n+1-i} + \hat{R}_i)$; `reserve_totals()` sums them. For the ccODP, (5.1) is the chain-ladder reserve by Section 2.7; for the bCCNN it is the sum of (3.5) over F at the chosen step. There are no standard errors in this strand of the code (se and cv are NA); companion III discusses how the uncertainty of (5.1) can be obtained.

5.3 DISPERSION OF THE BCCNN

A bCCNN has 547 parameters and is trained on 210 cells, so the GLM estimator (2.15), with $n_{obs} - n_{par}$ in the denominator, does not exist for it. Härkönen notes exactly this for her networks: "the number of parameters [...] is even larger than the number of observations which implies that the estimated overdispersion parameter cannot be computed according to Pearson statistics" (Härkönen, 2021: 53). Paper C (Section 3.3.2 and its Table 2) uses a heuristic instead: the ccODP dispersion is reduced by the relative decrease of the *validation* loss that the network achieved,

$$\hat{\phi}^{bCCNN} = \hat{\phi}_D \left(1 - \max(0, \Delta_V) \right), \Delta_V = 1 - \frac{D_V(t^*)}{D_V(0)}, \quad (5.2)$$

where $D_V(t)$ is the validation deviance after t steps (companion II, Section 4.2). The logic is that the deviance is an estimate of ϕ times the degrees of freedom; if the network explains a fraction Δ_V of the out-of-sample deviance, the unexplained variance is reduced by that fraction. The cut at zero means that a network that does not beat the chain ladder on the validation cells keeps the dispersion of the chain ladder

(Paper C sets 0% for its LoBs 3 and 6). The rule is `bccnn_phi(phi_odp, decrease_vali)`, and it feeds the Pearson residuals of the bCCNN.

5.4 DIAGNOSTICS COMPUTED FROM THE MEANS

5.4.1 Pearson residuals

The Pearson residuals (Paper C, Figure 7; `pearson_residuals()`) are computed on the true lower triangle in the script:

$$r_{i,j} = \frac{y_{i,j} - \mu_{i,j}}{\sqrt{\hat{\phi} \mu_{i,j}}}, \quad (5.3)$$

with the model's own $\hat{\phi}$, namely (2.15) for the ccODP and (5.2) for the bCCNN. Under (2.1) they have mean 0 and variance 1 in the fitted cells; on the lower triangle they are out-of-sample standardised errors.

5.4.2 Cumulative development factors

The cumulative development factors follow Paper C, Figure 8, and are computed by `cum_dev_factors()`. From a mean square μ , the individual factor of accident period i from development period $j - 1$ to j and its cumulative version are

$$f(i, j) = \frac{\sum_{l \leq j} \mu_{i,l}}{\sum_{l \leq j-1} \mu_{i,l}}, g(i, j) = \prod_{l=2}^j f(i, l) = \frac{\sum_{l \leq j} \mu_{i,l}}{\mu_{i,1}}, \quad (5.4)$$

$$j = 2, \dots, n.$$

Under the ccODP, $\mu_{i,l} = \hat{a}_i \hat{b}_l$ gives $g(i, j) = S_j / \hat{b}_1 = S_j$ (since $\hat{\beta}_1 = 0$), the same for every accident period, and by (2.14) it equals the product of the chain-ladder factors $\prod_{l=2}^j f_{l-1}$: every row of the ccODP factor matrix is the chain-ladder pattern. The numerical check finds the spread across rows at 3×10^{-15} . The bCCNN breaks the product form through the interaction term $\langle B, z^{(3)}(i, j) \rangle$, so its $g(i, j)$ vary with i : the plot of (5.4) against j , with one line per accident period, shows directly how far the network has let the development pattern depend on the accident period.

5.4.3 Relative difference

The relative difference (Paper C, Figure 5; `plot_relative_difference()`),

$$\frac{\mu_{i,j}^{bCCNN}}{\hat{\mu}_{i,j}^{cc}} - 1 = \exp \{ (w - 1) (\hat{\alpha}_i + \hat{\beta}_j) + (c - \hat{c}) + \langle B, z^{(3)}(i, j) \rangle \} - 1, \quad (5.5)$$

is the multiplicative correction of Section 3.5 minus one, shown as a heat map over the whole square. It is zero everywhere at step 0.

5.5 SUMMARY

For every model the reserve is the sum of the fitted means over the future cells, which for the ccODP is the chain-ladder reserve and for the bCCNN the sum of the network means at the chosen step. Because the bCCNN has more parameters than cells, its dispersion cannot be estimated by the GLM formula; Paper C's heuristic reduces the ccODP dispersion by the relative decrease of the validation deviance, cut at zero. The Pearson residuals, the cumulative development factors and the relative difference to the ccODP show where and how the network departs from the chain ladder; in particular, the ccODP has the same cumulative factors for every accident period, while the bCCNN lets them vary. Chapter 6 locates each formula in the code and reports the numerical check.

CHAPTER 6

IMPLEMENTATION AND VERIFICATION

6.1 INTRODUCTION

Chapters 2 to 5 stated the formulas of the bCCNN. This chapter links each of them to the function or line of code that computes it and reports the independent numerical check of the formulas described in Chapter 1. It closes with the summary and conclusions of the whole report.

6.2 WHERE EACH FORMULA IS IN THE CODE

Table 6.1 lists, for each formula or group of formulas, the place in the code where it is computed or asserted.

Table 6.1 **Location of the formulas in the code**

Formula	Where in the code
(1.1) to (1.3) cells, observed set, reserve to predict	triangle_sets(), fut_mask(), upper(), fut_cells() in R/triangles.R
(2.1) to (2.3) ccODP model and identification	fit_odp_glm(): glm(y ~ origin + dev, family = quasipoisson(link = "log")) with factors; alpha <- c(0, coef[origin2..n]), beta <- c(0, coef[dev2..n])
(2.4) Poisson deviance	poisson_deviance() in R/loss functions.R
(2.8) to (2.10) MLE by IWLS	Inside stats::glm(), glm.control(epsilon = 1e-12, maxit = 100)
(2.11) chain-ladder equivalence	Asserted in bCCNN fit.R: all.equal(res\$odp\$reserve * scale, cl\$total[["ibnr"]]), with fit_chainladder() in R/classical_models.R
Section 2.10, periods without payments	fit_odp_glm(): zero_origin, zero_dev; mask_zero_periods() in R/nn_models.R
(2.15) dispersion	fit_odp_glm(): phi_deviance, phi_pearson, n_par <- 2L * n - 1L
(2.16) unit of the payments	fit_odp_glm(scale =), dat_cfg\$scale; every output in those units
(3.1) and (3.2) embeddings and skip connection	layer_embedding(input_dim = n, output_dim = 1, trainable = ...), set_weights(list(as.matrix(odp\$alpha))), layer_add(..., name = "cc0")
(3.3) and (3.4) feed-forward part	layer_concatenate(name = "concat0"), layer_dense(units = q[h], activation = "tanh"), layer_dropout(rate = dropout)
(3.5) output, Paper C's equation (13)	layer_concatenate(name = "concat1"), layer_dense(units = 1, activation = "exponential", name = "Response")

Formula	Where in the code
(3.7) the start, Paper C's equation (14)	<code>set_weights(list(as.matrix(c(1, rep(0, q[3]))), array(odp\$c)))</code> on Response; <code>clear_session()</code> , <code>set_random_seed(seed)</code>
(3.8) and (3.9) Keras loss and deviance	<code>compile(loss = "poisson"); keras_poisson_to_deviance()</code>
Table 3.2, history	<code>fit_bccnn(): record()</code> , <code>callback_lambda(on_epoch_end = ...)</code> , <code>predict_on_batch(x_all)</code>
(4.1) to (4.4) gradients	Keras automatic differentiation; written out in this report only
(4.5) RMSprop	<code>optimizer_rmsprop(learning_rate, rho, epsilon)</code> with <code>config.yml</code> <code>bccnn: training:</code>
(5.1) reserves	<code>reserve_by_origin()</code> , <code>reserve_totals()</code> in <code>R/metrics.R</code> , <code>R/classical_models.R</code>
(5.2) bCCNN dispersion	<code>bccnn_phi()</code> ; <code>loss_decrease()</code> for Δ_V
(5.3) to (5.5) diagnostics	<code>pearson_residuals()</code> , <code>cum_dev_factors()</code> in <code>R/metrics.R</code> ; <code>plot_relative_difference()</code> in <code>R/plots.R</code>

6.3 NUMERICAL CHECK OF THE FORMULAS

The check script (`bccnn_check.py`, kept with the companion explainers) builds a 20×20 triangle from a known ccODP structure plus an interaction term that the ccODP cannot represent, draws over-dispersed Poisson payments in millions, and then recomputes everything above in numpy. The data are synthetic; they are there to test the formulas, not to say anything about the simulated portfolio. Table 6.2 lists the results.

Table 6.2 Results of the numerical check (amounts in millions of the synthetic data)

Check	Result
IWLS (2.10): chain-ladder reserve against ccODP reserve (2.11)	Relative difference 5.1×10^{-15}
Marginal totals (2.9): largest row sum and column sum of $\hat{\mu} - y$	$7.8 \times 10^{-12}, 6.7 \times 10^{-12}$
Chain-ladder factors from the data against S_k/S_{k-1} of the fitted means (2.14), first five	1.68757, 1.28717, 1.17237, 1.10129, 1.06176, both
Unit (2.16): $\hat{c}' - \hat{c}$ against $-\log u, u=1000$	-6.9077552790 against -6.9077552790 ; $\max \hat{\alpha}' - \hat{\alpha} = 7 \times 10^{-15}, \max \hat{\beta}' - \hat{\beta} = 7 \times 10^{-13}$
Cumulative factors (5.4) of the ccODP: spread across accident periods	3.1×10^{-15}
Dispersion (2.15): $\hat{\phi}_D, \hat{\phi}_P$ (true $\phi=0.4$ in these units; the misspecified interaction inflates both)	0.637, 0.622
Parameters (Table 3.1): trainable, non-trainable	547, 40
Property 1 (3.7): $\max_{i,j} \mu_{i,j}(\theta_0)/\hat{\mu}_{i,j}^{cc} - 1 $	1.4×10^{-14} ; reserve and deviance at step 0 equal those of the ccODP to all printed digits
Gradients (4.2) to (4.4) against central finite differences at the start: B (largest entry 2.08)	Absolute error 1.1×10^{-7} ; c and w : analytic 10^{-12} , finite difference 10^{-8} (both zero to the accuracy of the difference quotient); hidden layers: exactly 0, finite differences exactly 0
Gradients at a random point ($B \neq 0$): largest relative error by group	$W^{(1)} 1.1 \times 10^{-7}, b^{(1)} 9 \times 10^{-10}, W^{(2)} 2.4 \times 10^{-8}, b^{(2)} 3.7 \times 10^{-9}, W^{(3)} 1.1 \times 10^{-6}, b^{(3)} 6.8 \times 10^{-8}, B 2.4 \times 10^{-8}, w 3 \times 10^{-11}, c 1.5 \times 10^{-10}$
Keras loss identity (3.9), $N=210$: $2N\ell + 2\sum(y \log y - y)$ against direct deviance	108.966655469307 against 108.966655469310; effect of the ε of Keras on ℓ : 9.7×10^{-8}
Property 3 (4.6): largest first-step update, B against $\eta/\sqrt{1-\rho}$; all other groups	0.0031621 against 0.0031623; hidden layers 0, $w 1.1 \times 10^{-10}, c 1.8 \times 10^{-10}$
Training deviance over the first three RMSprop steps (without dropout)	108.967, 109.034, 109.198, 108.973

6.4 SUMMARY AND CONCLUSIONS

The report set out the mathematics behind the bCCNN as the code implements it, and every formula was located in the code (Table 6.1) and confirmed by an independent numerical check (Table 6.2). Five conclusions follow.

First, the ccODP model fitted by minimising the Poisson deviance reproduces every observed row and column total of the triangle, and these marginal totals alone make its reserve the chain-ladder reserve and its development pattern the chain-ladder pattern. The dispersion does not enter the point estimates, and the unit of the payments shifts only the intercept.

Second, the bCCNN embeds the ccODP as a sub-network: the frozen embeddings hold the ccODP effects, and the skip connection carries the ccODP linear predictor to the output. The feed-forward part sees only the pair of effects of a cell, so what the network adds is a multiplicative correction of the chain-ladder means that depends on the accident-period and development-period effects jointly, that is, an interaction that lets the development pattern vary with the accident period.

Third, the loss that Keras minimises is the Poisson deviance up to a constant and the factor $2N$, so the network is fitted by the same criterion as the ccODP. At the start θ_0 of Paper C's equation (14) the network reproduces the ccODP means exactly; the marginal-total equations make the gradients in the intercept and the skip weight vanish, and the zero output weights block the hidden layers, so the first step changes only the output weights of the ten random hidden features. Because the first RMSprop step is sign-like, these weights move by about 0.00316 each whatever the size of the gradient, and the training deviance can rise for a few steps before it falls. The number of steps therefore measures how far the network is allowed to move from the chain ladder, and zero steps return the chain ladder itself.

Fourth, the reserves and diagnostics follow directly from the fitted means, but the dispersion of the bCCNN cannot be estimated by the GLM formula, because the network has more parameters than cells; the code uses Paper C's heuristic based on the decrease of the validation deviance.

Fifth, periods without payments in the fitted cells lead to effects of about -30 , which are harmless for the chain ladder but can dominate a validation or test deviance. Neither Paper C nor Härkönen (2021) treats this case; the code masks the affected cells out of the deviance and leaves the network input unchanged. Two points therefore remain open for further work: the input of the network for such periods, and an estimator of the dispersion of the bCCNN that rests on the actual number of parameters rather than on a heuristic, which Härkönen (2021: 53) also leaves for future work. The use of these formulas in the fitting procedure is the subject of companion II, and their interpretation and evaluation that of companion III.

References

- Al-Mudafer, M.T., Avanzi, B., Taylor, G. & Wong, B. 2021. Stochastic loss reserving with mixture density neural networks. arXiv preprint 2108.07924.
- Department of Statistics and Actuarial Science. 2026. *Research assignment guide*. Last updated 28 January 2026. Stellenbosch: Stellenbosch University.
- Gabrielli, A., Richman, R. & Wüthrich, M.V. 2020. Neural network embedding of the over-dispersed Poisson reserving model. *Scandinavian Actuarial Journal*, 2020(1):1-29.
- Gabrielli, A., Richman, R. & Wüthrich, M.V. 2018. Neural network embedding of the over-dispersed Poisson reserving model. SSRN working paper 3288454, version of 21 November 2018.
- Goodfellow, I., Bengio, Y. & Courville, A. 2016. *Deep learning*. Cambridge, MA: MIT Press.
- Hachemeister, C.A. & Stanard, J.N. 1975. IBNR claims count estimation with static lag functions. Paper presented at the ASTIN Colloquium. Portimão, Portugal.
- Härkönen, V. 2021. On claims reserving with machine learning techniques. Unpublished master's thesis. Stockholm: Stockholm University.
- Mack, T. 1991. A simple parametric model for rating automobile insurance or estimating IBNR claims reserves. *ASTIN Bulletin*, 21(1):93-109.
- McCullagh, P. & Nelder, J.A. 1989. *Generalized linear models*. 2nd edition. London: Chapman and Hall.
- Renshaw, A.E. & Verrall, R.J. 1998. A stochastic model underlying the chain-ladder technique. *British Actuarial Journal*, 4(4):903-923.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. 2014. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929-1958.
- Wüthrich, M.V. & Merz, M. 2019. Editorial: Yes, we CANN! *ASTIN Bulletin*, 49(1):1-3.

APPENDIX A

LIST OF SYMBOLS

The symbols used in this report are listed in Table A.1, with the part of the report where each is introduced.

Table A.1 Symbols used in this report

Symbol	Meaning	Where introduced
n	Number of accident periods = number of development periods; here $n = 20$ (annual)	Section 1.2
i, j	Accident period $1, \dots, n$ and development period $1, \dots, n$ (1-based, as in the code; Paper C's development year j is $j - 1$ here)	Section 1.2
$k = i + j - 1$	Calendar period of cell (i, j) ; the valuation date is calendar period n	Section 1.2
O, F	Observed cells $\{i + j \leq n + 1\}$ (210 of them) and future cells $\{i + j > n + 1\}$ (190)	Section 1.2
$Y_{i,j}, y_{i,j}$	Incremental payments of cell (i, j) as a random variable and as observed, in units of u	Section 1.2
u	Unit of the payments in the fit, $u = 10^6$ (data: scale)	Section 2.9
c, α_i, β_j	Intercept, accident-period effect and development-period effect of the ccODP model, $\alpha_1 = \beta_1 = 0$	Chapter 2
$\hat{c}, \hat{\alpha}_i, \hat{\beta}_j$	Their maximum likelihood estimates	Chapter 2
$\mu_{i,j}$	Mean of cell (i, j) under the model at hand; $\hat{\mu}_{i,j}^{cc}$ for the fitted ccODP	Chapters 2 and 3
ϕ	Dispersion: $\text{Var}(Y_{i,j}) = \phi \mu_{i,j}$	Chapter 2
$D(y, \mu; S)$	Poisson deviance of the means μ on the cells S	Section 2.4
n_{obs}, n_{par}	Number of fitted cells and of ccODP parameters, $n_{par} = 2n - 1 = 39$	Section 2.3
f_j	Chain-ladder factor from development period j to $j + 1$	Section 2.7
q_1, q_2, q_3	Neurons of the three hidden layers, $(20, 15, 10)$	Chapter 3
$W^{(h)}, b^{(h)}$	Weights ($q_{h-1} \times q_h$) and biases of hidden layer h , $q_0 = 2$	Chapter 3

Symbol	Meaning	Where introduced
$z^{(h)}(i, j)$	Activations of hidden layer h for cell (i, j) ; $z^{(0)} = (\alpha_i, \beta_j)^T$	Chapter 3
p	Dropout rate, $p = 0.1$; $m^{(h)}$ a dropout mask	Section 3.6
w, c, B	Weight of the skip connection, intercept and weights of the output neuron	Section 3.5
θ	All trainable parameters, $\theta = (W^{(1)}, b^{(1)}, \dots, B, w, c)$; 547 numbers	Chapter 3
θ_0	The start of gradient descent, Paper C's equation (14)	Section 3.7
$a_{i,j}$	Output before the exponential, $a = \log \mu$	Section 3.5
$\ell(\theta)$	The loss Keras minimises (mean Poisson loss over the training cells)	Section 3.8
N	Number of training cells, $N = 210$ on the full triangle	Section 3.8
$\delta_{i,j}$	Derivative of the loss in the output $a_{i,j}$	Chapter 4
η, ρ, ϵ	RMSprop learning rate, decay and stabiliser: 0.001, 0.9, 10^{-7}	Section 4.6
$\hat{R}_i, \hat{R}$	Reserve (predicted outstanding payments) of accident period i and in total	Section 5.2
$g(i, j)$	Cumulative development factor of accident period i to development period j	Section 5.4